USING SIMULATED ANNEALING TO APPROXIMATE A GOOD SOLUTION TO THE TRAVELLING SALESMAN PROBLEM

BY

ANANTA THAPALIYA

Wake Forest University

Winston Salem, North Carolina
December 2020

Submitted To

Dr. Sami Khuri

CSC721: Algorithms

ABSTRACT

The Travelling Salesman Problem (TSP) problem is an NP-Complete problem for which a polynomial time solution has not been discovered yet. However, various solutions with exponential runtimes do exist. For instance, TSP can be solved using dynamic programming in $O(n^2 2^n)$ runtime (Bellman, 1962). This is unrealistic for large input sizes. Therefore, computer scientists use heuristics, executable in reasonable CPU/GPU time, in order to aim for a good solution instead of the best (optimal) solution for the travelling salesman problem. Simulated Annealing (SA) is one of such heuristics. It is a probabilistic technique using which a good solution to TSP can be approximated. This article implements and applies the Simulated Annealing algorithm to three TSP datasets obtained from the internet (Reinelt, 1995).

.

KEYWORDS AND ABBREVIATIONS

Keywords: graph theory, travelling salesman problem, simulated annealing, NP completeness

Abbreviations

TSP	Travelling Salesman Problem
CPU	Central Processing Unit
GPU	Graphics Processing Unit
SA	Simulated Annealing

LIST OF TABLES

		Page
Table 1	Results Summary	11

LIST OF FIGURES

		Page
Figure 1.	Simulated Annealing Setup for TSP	4
Figure 2.	Cost Curve for FRI26 Dataset (26 Cities)	5
Figure 3.	Hamiltonian Tour Optimized by Simulated Annealing Algorithm (fri26 dataset)	6
Figure 4.	Cost Curve for DANTZIG42 Dataset (42 Cities)	7
Figure 5.	Hamiltonian Tour Optimized by Simulated Annealing Algorithm (dantzig42)	8
Figure 6.	Cost Curve for ATT48 Dataset (48 Cities)	9
Figure 7	Fig 7: Hamiltonian Tour Optimized by Simulated Annealing Algorithm (att48)	10

TABLE OF CONTENTS

ABSTRACT **II**

KEYWORDS AND ABBREVIATIONS **III**

LIST OF TABLES **IV**

LIST OF FIGURES **V**

TABLE OF CONTENTS **VI**

BACKGROUND **1**

TRAVELLING SALESMAN PROBLEM (TSP) 1
SIMULATED ANNEALING (SA) 1

METHODOLOGY **3**

RESULTS **5**

FRI26 DATASET 5
DANTZIG42 DATASET 7
ATT48 DATASET 9

DISCUSSION **12**

CONCLUSION 12
LIMITATIONS AND FUTURE WORKS 13

REFERENCES **15**

BACKGROUND

Travelling Salesman Problem (TSP)

According to Richard Bellman (1962):

> The well-known travelling salesman problem is the following: "A salesman is required to visit once and only once each of n different cities starting from a base city and returning to this city. What path minimizes the total distance travelled by the salesman?" (p. 61)

TSP involves finding the minimum-cost Hamiltonian tour in the given graph. Therefore, the objective function to minimize is the sum of all edge-weights present in the given Hamiltonian tour instance. In our case, we will assume that there is a direct path between any two cities which makes the graph complete. The dynamic programming solution to TSP for n cities has a runtime of $O(n^2 2^n)$ (Bellman, 1962). The cost of a Hamiltonian circuit (HC) for a graph G with each edge *e* with a non-negative weight can be defined as:

$$C(HC) = \sum_{e \in HC} c(e), \text{ where } c(e) \text{ is the cost (weight) of the edge}$$

This is the objective function that should be minimized by the Simulated Annealing Algorithm.

Simulated Annealing (SA)

Simulated Annealing is a randomized technique used for optimization problems. The idea was derived from a technique called annealing used in the field of metallurgy. Imaginary physical systems in near-optimal configuration are usually in low energy state (Kirkpatrick, 1984). In metallurgy, the metal is heated to a very high temperature in order to allow the molecules to rearrange themselves randomly. Then it is cooled down out slowly in order to allow

the molecules to reach lower energy states and eventually reach a near-optimal and more stable configuration. This concept can be applied to optimization problems like TSP.

Let us suppose,

T_0 = starting temperature of the system

C_i = current configuration of the system

C_n = next (altered) configuration of the system

α = temperature change

J(C) = cost function for the given configuration

We initially start with a random initial configuration $C_i = C_0$ (a randomly chosen Hamiltonian tour in our case), a high temperature $T = T_0$, and a cooling rate α. These are the hyperparameters that have to be set before starting the process. Then we run the following algorithm for several epochs. The number of epochs is also a hyper parameter.

1. We randomly (or other ways) alter the configuration of the system from C_i to the next configuration C_n
2. We check if $J(C_n) <= J(C_i)$
    a. if yes, we choose C_n
    b. if no, $\Delta J = J(C_i) – J(C_n)$
        i. calculate the probability $P = e^{\Delta J/T}$
    c. if P is realistic, we choose C_n, else we keep C_i
    d. $T = T – \alpha$

This is the general framework for SA algorithm based on Bertsimas's work (1993). We will apply a flavor of this algorithm to approximate a good solution for TSP.

METHODOLOGY

Three datasets each with 26, 42, and 48 cities were chosen (Reinelt, 1995). A python program was implemented to perform SA, for which the generic form of SA discussed in the previous section was taken as a reference. The initial configuration can be thought of as any feasible Hamiltonian tour for the given complete graph. For four cities (A, B, C, and D), for instance, a feasible route the travelling salesman can take is ABCD-A. The tour has to end at the starting city; so, city A is visited twice. Therefore, the cost function $J(C_k)$ for any configuration k returns the cost of the Hamiltonian tour represented by C_k. The SA's job is to minimize the cost.

$J(C_k)$ = path weight or cost of the Hamiltonian tour

In order to alter the configuration and simulate the "cooling" process, the positions of two random cities in the current configuration were switched. However, doing so, the starting city and the ending city were left unaltered with a constrain that the salesman must start from the given starting city. If the current configuration was ABCD-A, any two nodes from BCD would be switched in order to move to the next configuration leaving city A unchanged on both ends.

The decision to accept or reject the altered configuration obtained by this method was driven by the cost function J. In every epoch, if the cost of this altered (next) configuration (C_n) was less than that of the initial configuration (C_i), C_n was accepted as the current configuration. If that was not the case, then either C_i would persist, or C_n would be accepted based on the probability equation explained in the background. If this probability was better than a random probability (a random floating-point value between 0 and 1) then C_n would be accepted. The temperature was also decreased by α every epoch. Figure 1 demonstrates the TSP setup for this project based on Bertsimas's work (1993) and Badri Adhikari's explanation of Simulated Annealing Algorithm (2017).

The cost curves were plotted for all three datasets using a Python library called matplotlib (Hunter, 2007). Additionally, the graph representations of all three datasets showing the SA optimized routes were also visualized using another Python library called networkx (Hagberg, 2008). The cost of thus obtained optimized Hamiltonian tour was compared to the real optimal solution provided by the same source the datasets were obtained from (Reinelt, 1995). The results of this experiment are presented and discussed in the following sections.

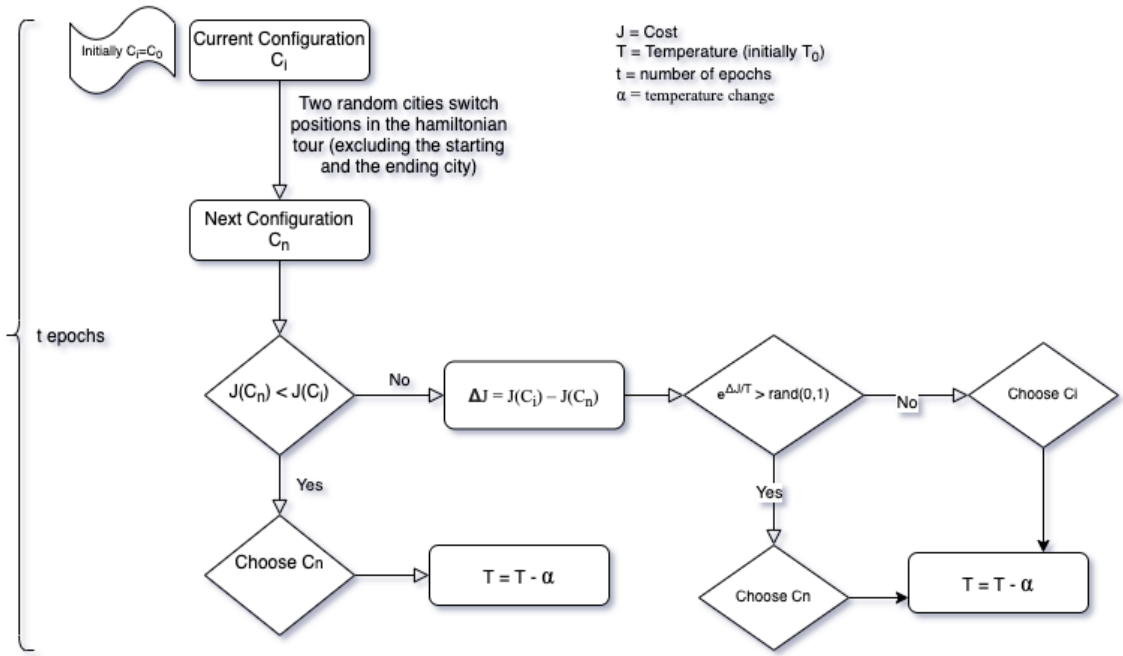

Fig 1: Simulated Annealing Setup for TSP

RESULTS

FRI26 Dataset

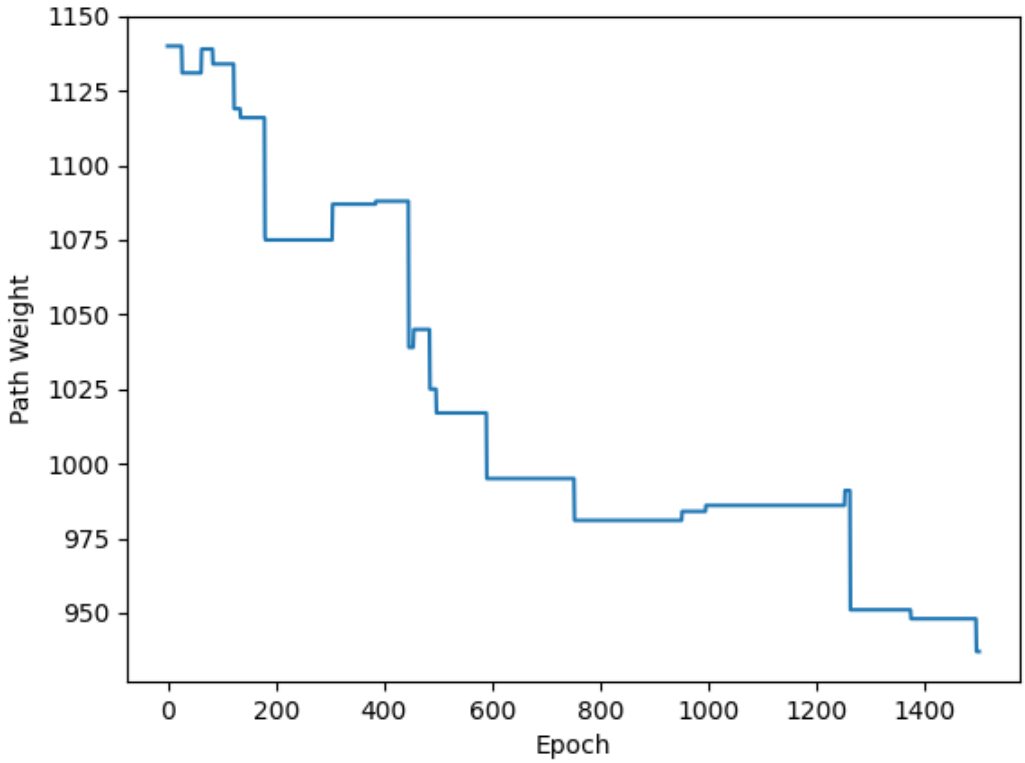

Fig 2: Cost Curve for FRI26 Dataset (26 Cities)

Figure 2 shows the decreasing cost while SA was being applied to the fri26 dataset comprising 26 cities (Reinelt, 1995). Figure 3 shows the graph representation of this dataset and the Hamiltonian tour optimized by the Simulated Annealing algorithm described in the methodology. This particular simulation was started at an initial temperature T = 5 for 1500 epochs. The temperature was decreased by 0.001 after every epoch.

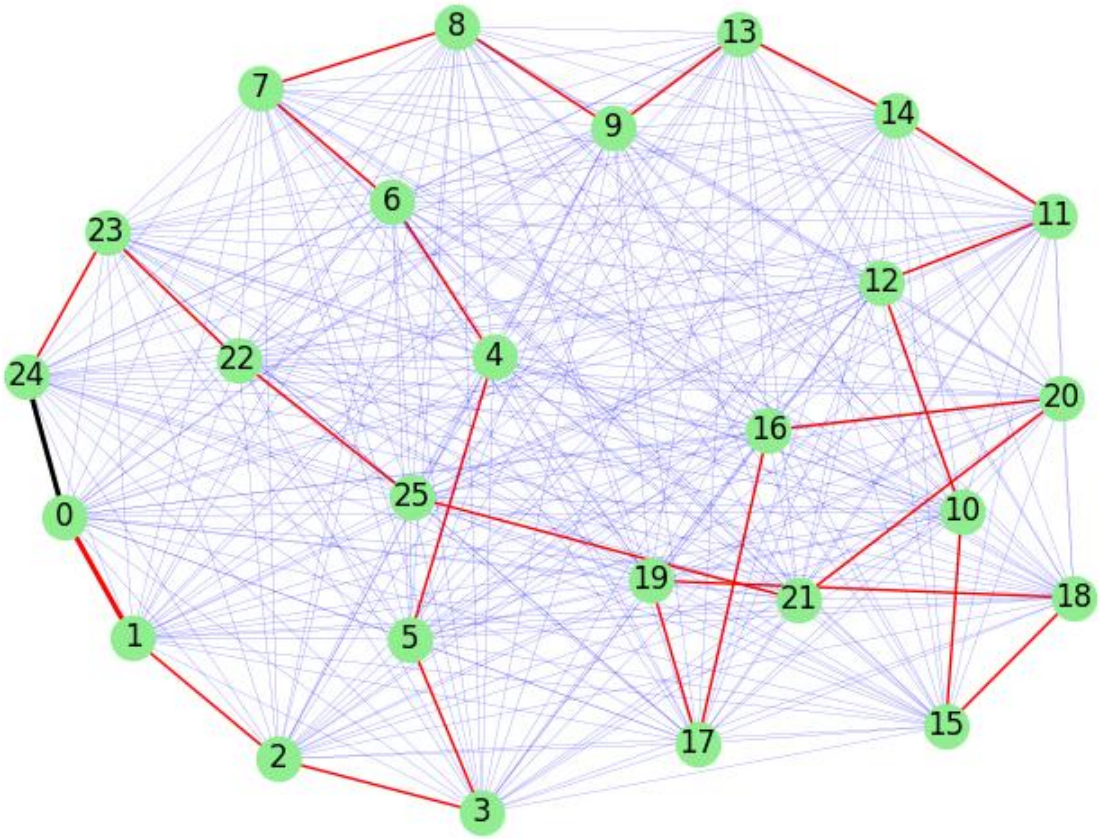

Fig 3: Hamiltonian Tour Optimized by Simulated Annealing Algorithm (fri26 dataset)

The cost of the Hamiltonian tour optimized by SA for fri26 dataset was found to be 937. The real optimal solution for this dataset is also 937. (Reinelt, 1995). The dark edges in figure 3, 5, and 7 represent the edge that ended the tour by returning to node 0. Similarly, the dark red edge represents the first hop taken from the starting node 0.

DANTZIG42 Dataset

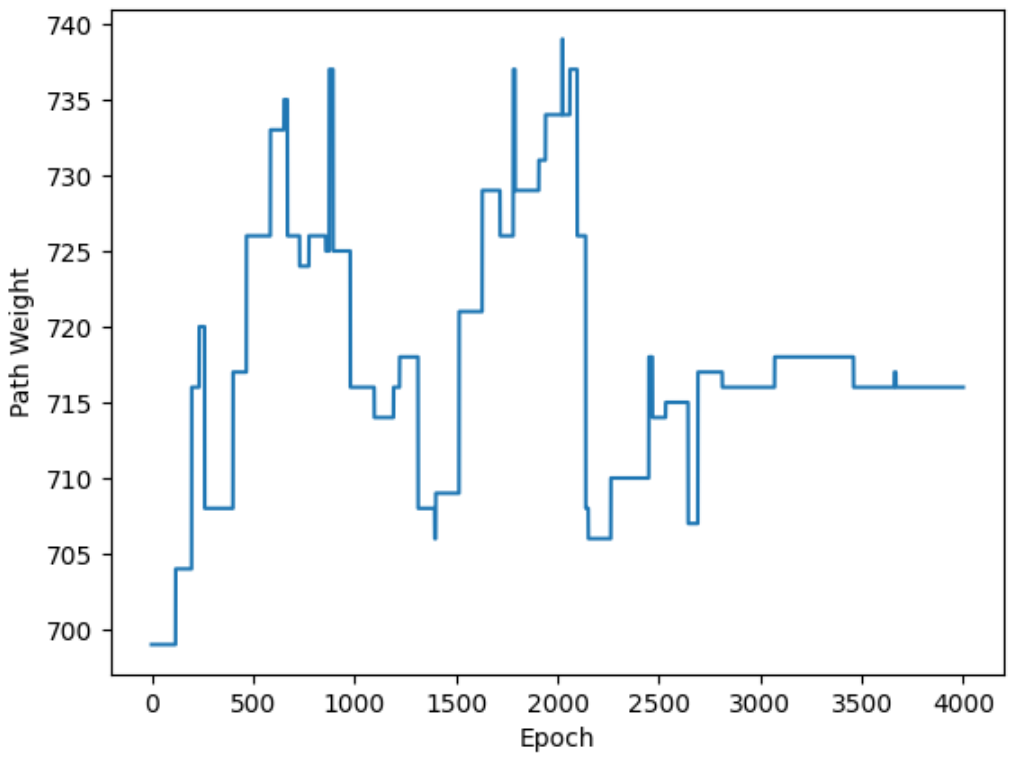

Fig 4: Cost Curve for DANTGIZ42 Dataset (42 Cities)

Figure 4 shows the changing cost while SA was being applied to the dantzig42 dataset comprising 42 cities (Reinelt, 1995). Figure 5 shows the graph representation this dataset and the Hamiltonian tour optimized by the Simulated Annealing algorithm described in the methodology. This particular simulation was started at an initial temperature T = 8 for 4000 epochs. The temperature was decreased by 0.001 after every epoch.

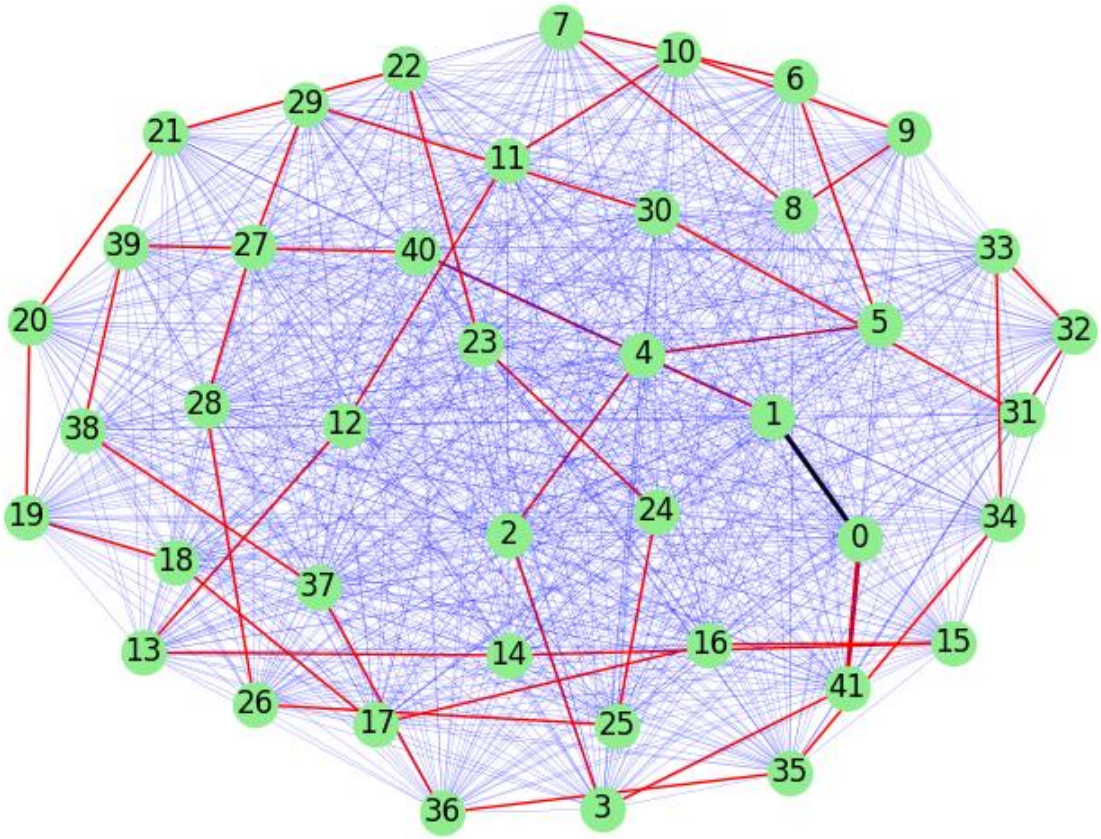

Fig 5: Hamiltonian Tour Optimized by Simulated Annealing Algorithm (dantzig42)

The cost of the Hamiltonian tour optimized by SA for dantzig42 dataset was found to be 716. The real optimal solution for this dataset is 699 (Reinelt, 1995).

ATT48 Dataset

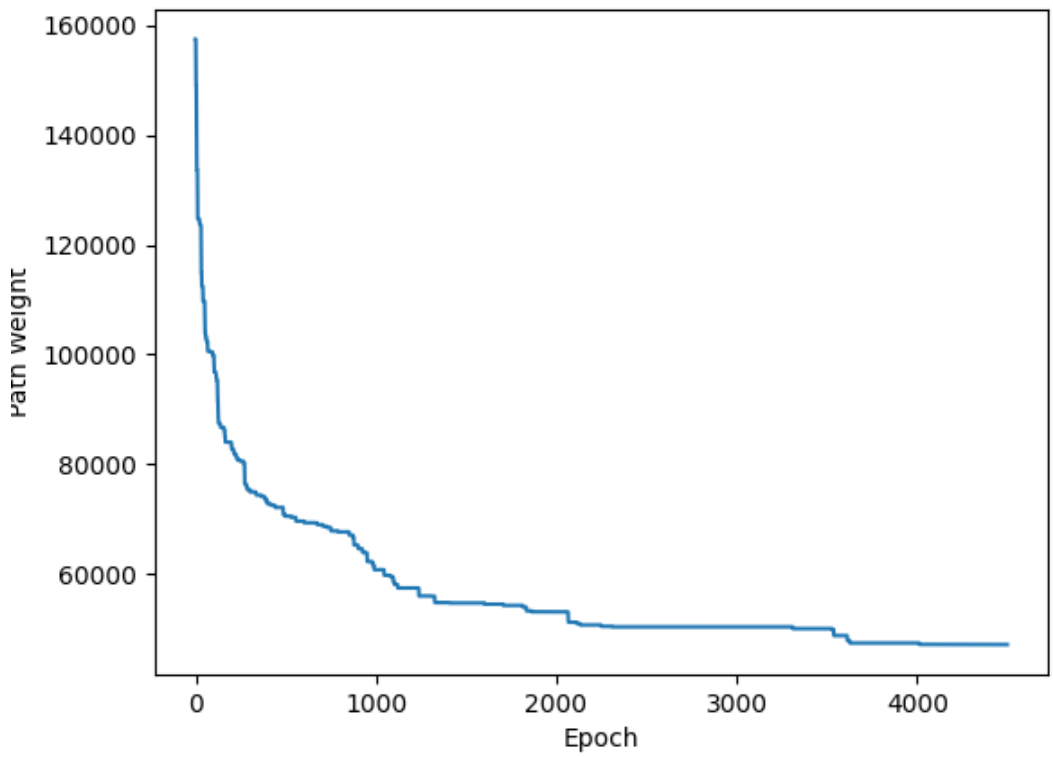

Fig 6: Cost Curve for ATT48 Dataset (48 Cities)

Figure 6 shows the decreasing cost while SA was being applied to the att48 dataset comprising 48 US capitols (Reinelt, 1995). Figure 7 shows the graph representation of this dataset and the Hamiltonian tour optimized by Simulated Annealing algorithm described in the methodology. This particular simulation was started at an initial temperature T = 13 for 4500 epochs. The temperature was decreased by 0.001 after every epoch.

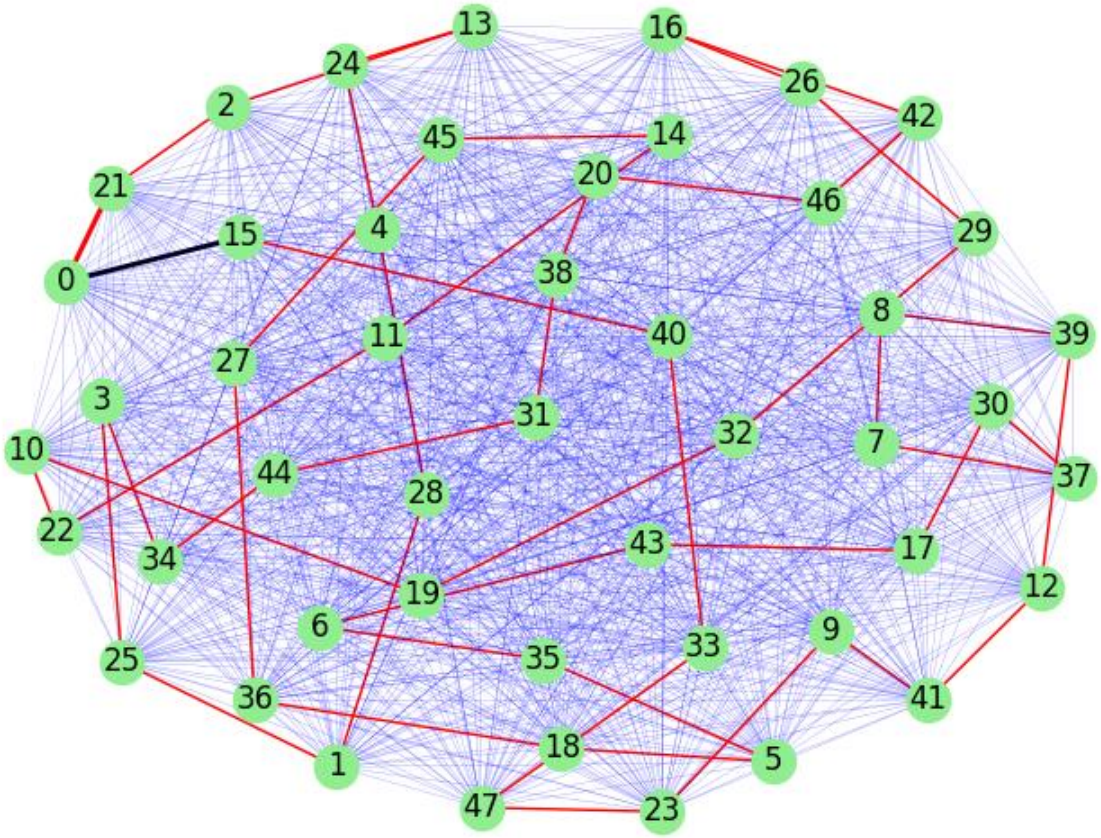

Fig 7: Hamiltonian Tour Optimized by Simulated Annealing Algorithm (att48)

The cost of the Hamiltonian tour optimized by SA for att48 was found to be 47084. The real optimal solution for this dataset is 33523 (Reinelt, 1995).

Table 1: Results Summary

TSP Dataset	**True Optimal Solution**	**SA Approximated Solution**	**Hyperparameters** (α = 0.001)
fr26 26 cities (nodes)	937	937 difference = 0	T_0 = 5, epochs = 1500
dantzig42 42 cities (nodes)	699	716 difference = 17	T_0 = 8, epochs = 4000
att48 48 cities (nodes)	33523	47984 difference = 14461	T_0 = 8, epochs = 4500

Table 1 compares the real optimal Hamiltonian tour to the SA-approximated tour for the three datasets chosen for this project. The following section discusses the implications of these results.

DISCUSSION

Conclusion

The goal of implementing Simulated Annealing (SA) algorithm was to get a good solution if not the best (optimal) solution for TSP. SA is a heuristic that allows us to approximate the solution for TSP faster in a stochastic manner although there is no way to verify if the solution is optimal or to know how good the solution is. Fortunately, for the chosen datasets, the optimal solutions were already made available by the source (Reinelt, 1995), which made it easier to compare the output of SA to the real solutions. The following observations were made from the experiments conducted with fri26, dantzig42, and att48 datasets.

1. **Correctness:** We cannot be too optimistic about the correctness of the solution when we use randomized approaches. However, with a good heuristic like SA, we can still achieve a "good enough" solution. Fri26 dataset performed very well with SA and easily produced the optimal solution. However, as the number of cities increased, it became increasingly difficult for the model to explore the solution space to find better solutions. Table 1 shows that, at least for these three datasets, as the number of cities increased, the approximated solutions deviated more from the real optimal solutions. The model performed the worst for att48, which is a realistic dataset of 48 US capitols.
2. **Goodness:** It is fair to say that SA still approximated a "good enough" solution for att48 by looking at the cost curve. The starting cost for att48, as shown by figure 6, was 120,000. This is how bad a randomly chosen Hamiltonian tour could be. SA managed to slowly explore the solution space and finally bring the cost down to 47084, which is pretty good compared to 120,000. It was off by only 14,461 from the optimal solution. The runtime for the optimal solution would have been O ($48^2 2^{48}$) using dynamic programming. SA, by

approximating it in 4500 epochs, still remains an amazing tour guide by suggesting a far more realistic and better route compared to a randomly selected one without taking ~$48^2 2^{48}$ CPU time.

3. **Trends**: The cost curve for dantzig42 (shown in figure 4) acted strangely. It actually started from a lower cost. It made a series of good and bad decisions along the way trying to bring the cost of the tour down. Still, it landed on a "good enough" position after 4500 epochs, deviating only by 17 from the true optimal solution (Table 1). This phenomenon is actually common in SA because this method accepts or rejects a given configuration based on probabilities. It might accept a bad move if what appears to be a bad move at the moment has a higher probability of being useful later to find better solutions. The cost curve for fri26 (figure 2) also shows this trend. A series of small fluctuations that slightly elevated the cost at the moment proved beneficial later by contributing to the convergence.

Limitations and Future Works

There are a few limitations of this project, the primary one being the size of the dataset. It would be interesting to see how this approach performs on datasets with more than 48 cities. Furthermore, the function designed to move from the current configuration (C_i) to the next configuration C_n) only randomly swaps the positions of two cities. This only slightly changes the node arrangement of the current Hamiltonian tour. Therefore, it took many epochs for the model to explore the solution space well. Future works can explore random shuffling as a way to alter the given configuration. This might add more randomness to the algorithm and allow the model to explore the solution space in a more robust fashion. The implementation also captures the entire simulation as an animated video of how different paths were chosen along the way. This implementation can be generalized to work with other heuristics as well, not just SA. Furthermore,

the animation can be made more elegant by integrating various other metrics. This is possible by exploring more advanced functionalities of the networkx library. Nevertheless, the code is still reusable for various graph visualization problems.

REFERENCES

1. Bellman, R. (1962). Dynamic programming treatment of the travelling salesman problem. *Journal of the ACM (JACM)*, *9*(1), 61-63.
2. Kirkpatrick, S. (1984). Optimization by simulated annealing: Quantitative studies. *Journal of statistical physics*, *34*(5-6), 975-986.
3. Bertsimas, D., & Tsitsiklis, J. (1993). Simulated annealing. *Statistical science*, *8*(1), 10-15
4. Reinelt, G. (1995). Tsplib95. *Interdisziplinäres Zentrum für Wissenschaftliches Rechnen (IWR), Heidelberg*, *338*, 1-16. Retrieved November 27, 2020 from http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/
5. Adhikari, Badri [Badri Adhikari]. (2017, Feb 4). *The Simulated Annealing Algorithm Explained with an Analogy to a Toy* [Video]. YouTube: https://www.youtube.com/watch?v=eBmU1ONJ-os
6. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, *9*(3), 90–95.
7. Hagberg, A., Swart, P., & S Chult, D. (2008). *Exploring network structure, dynamics, and function using NetworkX*.